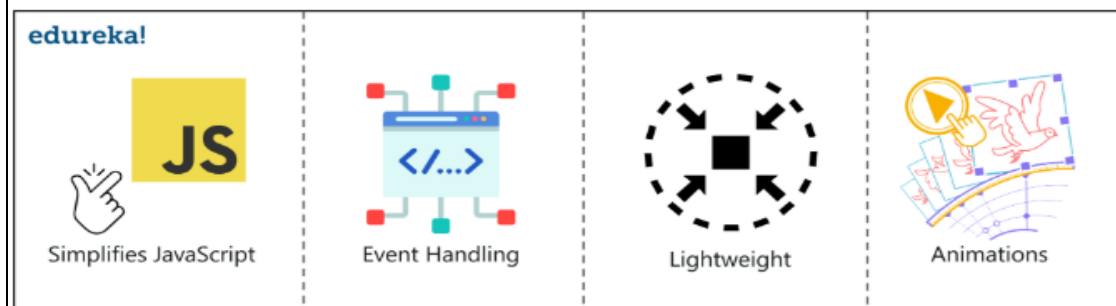


jQuery.

jQuery is a tiny JavaScript library that transforms the web into an entertaining experience. It is one of the most popular JavaScript libraries.

What is jQuery?

jQuery is an efficient & fast JavaScript Library created by John Resig in 2006. The motto of jQuery is write less, do more, which is very apt because its functionality revolves around simplifying each and every line of code. Below is a list of jQuery key features:



What is jQuery – jQuery Tutorial

- **Simplifies JavaScript:** It simplifies DOM manipulation and event handling for rapid web development
- **Event handling:** jQuery offers an effective manner to capture a wide variety of events, such as a user clicking on a link, without the need to clutter the HTML code
- **Lightweight:** jQuery is a compact, lightweight library of about 19KB in size
- **Animations:** It comes with plenty of built-in animation effects which you can use in your web app to make it more interactive

Why use jQuery?

- For starters, jQuery makes it extremely easy to manipulate the DOM. In order to **make a web app more interactive, web developers manipulate the DOM & jQuery** makes it very easy to do that.
- The community of its users is more diverse than any other JavaScript library. It **provides detailed documentation**, which is all a developer needs.

- jQuery has **1000s of plugins** available for free, which improve the user experience. One such example is the AJAX (Asynchronous JavaScript And XML), which develops a responsive and feature-rich site.
- jQuery provides **cross-browser support**, which lets you run your code on different **browsers** without worrying about dependency issues.

Install jQuery.

There's no installation per se; it's more like **downloading jQuery**. There are **two ways to install jQuery**:

- Local Installation:** You can **directly download the jQuery library** from its **official website** (<https://jquery.com/>) and include it in your HTML code.
- Link to a CDN:** You can **add the jQuery library** in your HTML code from a **Content Delivery Network**.

A. Local Installation:

There are **two versions of jQuery** available for downloading:

- **Production version (.min.js)** - this is for **your live website** because it has been **minified and compressed**.
- **Development version (.js)** - this is for **testing and development** (**uncompressed and readable code**).

Both versions can be downloaded from [jQuery.com](https://jquery.com).

The **jQuery** library is a **single JavaScript file**, and you reference it with the HTML **<script>** tag (notice that the **<script>** tag should be inside the **<head>** or **<body>** section):

```
<head>
<script src="jquery-3.4.1.min.js"></script>
</head>
```

B. Link to a CDN:

If you don't want to download and host jQuery yourself, you can **include it from a CDN (Content Delivery Network)**. Both Google and Microsoft host jQuery.

Google CDN:

```
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/jquery.min.js"></script>
</head>
```

[Try it Yourself »](#)

Microsoft CDN:

```
<head>
<script src="https://ajax.aspnetcdn.com/ajax/jquery/jquery-3.4.1.min.js"></script>
</head>
```

[Try it Yourself »](#)

➤ **Refer link for jQuery Get Started:**
https://www.w3schools.com/jquery/jquery_get_started.asp.

jQuery Syntax

The jQuery syntax is tailor-made for **selecting** HTML elements and performing some **action** on the element(s).

Basic syntax is: `$(selector).action()`

- A \$ sign to define/access jQuery
- A (selector) to "query (or find)" HTML elements
- A jQuery action() to be performed on the element(s)

Examples:

`$(this).hide()` - hides the current element.

`$("p").hide()` - hides all <p> elements.

`$(".test").hide()` - hides all elements with class="test".

`$("#test").hide()` - hides the element with id="test".

The Document Ready Event

You might have noticed that all jQuery methods in our examples, are inside a document ready event:

```
$(document).ready(function(){  
  
    // jQuery methods go here...  
  
});
```

This is to prevent any jQuery code from running before the document is finished loading (is ready).

It is good practice to wait for the document to be fully loaded and ready before working with it. This also allows you to have your JavaScript code before the body of your document, in the head section.

Here are some examples of actions that can fail if methods are run before the document is fully loaded:

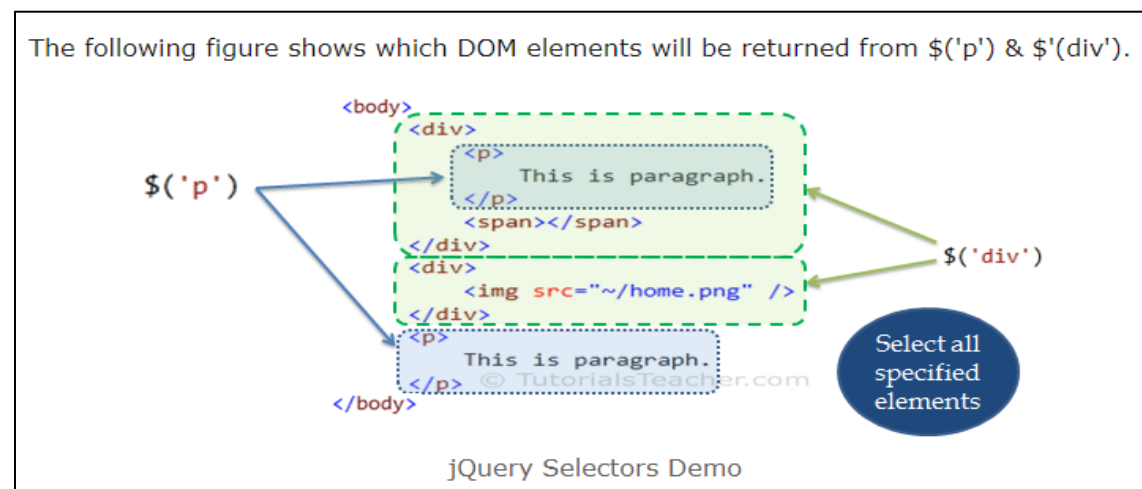
- Trying to hide an element that is not created yet
- Trying to get the size of an image that is not loaded yet

Tip: The jQuery team has also created an even shorter method for the document ready event:

```
$(function(){  
  
    // jQuery methods go here...  
  
});
```

jQuery Selectors.

- jQuery selectors allow you to **select and manipulate HTML elements (s)**.
- jQuery selectors are used to **"find" (or select) HTML elements based on their name, id, classes, types, attributes, values of attributes, and much more**. It's based on the existing CSS Selectors, and in addition, it has its custom selectors.
- All selectors in **jQuery start with the dollar sign and parentheses: \$()**.



Basic Selectors

Selector	Example	Description
*	\$("*")	Selects all elements.
#id	\$("#foo")	Selects all elements with the <code>id="foo"</code> .
.class	\$(".bar")	Selects all elements with the <code>class="bar"</code> .
element	\$("p")	Selects all <code><p></code> elements.
selector1, selector2, selectorN	\$("h1, p.bar, span")	Selects all <code><h1></code> and <code></code> , but only that <code><p></code> elements that has the <code>class="bar"</code> .

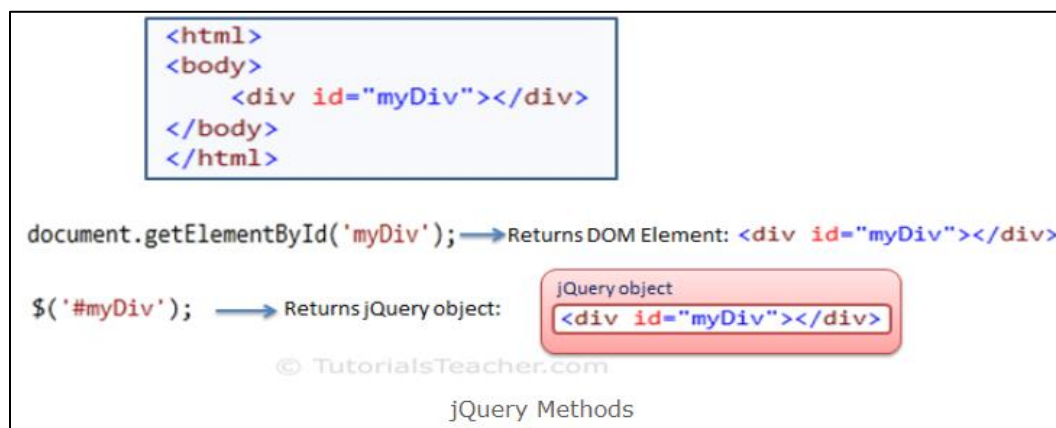
Basic Filter Selectors

Selector	Example	Description
:first	<code>\$("p:first")</code>	Selects the first <code><p></code> element.
:last	<code>\$("p:last")</code>	Selects the last <code><p></code> element.
:even	<code>\$("tr:even")</code>	Selects all even <code><tr></code> elements, zero-indexed.
:odd	<code>\$("tr:odd")</code>	Selects all odd <code><tr></code> elements, zero-indexed.
:eq()	<code>\$("tr:eq(1)")</code>	Select the 2nd <code><tr></code> element within the matched set, zero-based index.
:not()	<code>\$("p:not(:empty)")</code>	Select all <code><p></code> elements that are not empty.

- Refer [link](https://www.w3schools.com/jquery/jquery_selectors.asp) for jQuery Selectors:
- Refer link for jQuery Selectors: <https://www.tutorialsteacher.com/jquery/jquery-selectors>

jQuery Methods.

The jQuery selector finds particular DOM element(s) and wraps them with a jQuery object. For example, `document.getElementById()` in JavaScript will return a DOM object, whereas `$('#id')` will return a jQuery object. The following figure illustrates the difference.



The diagram illustrates the difference between a standard DOM method and a jQuery selector. At the top, a code block shows the HTML structure: `<html><body><div id="myDiv"></div></body></html>`. Below this, two methods are compared:

- `document.getElementById('myDiv');` is shown with an arrow pointing to the text "Returns DOM Element:" followed by the raw HTML snippet `<div id="myDiv"></div>`.
- `$('#myDiv');` is shown with an arrow pointing to the text "Returns jQuery object:". To the right, a red-bordered box labeled "jQuery object" contains the wrapped HTML snippet `<div id="myDiv"></div>`.

At the bottom, the text "© Tutorialsteacher.com" and "jQuery Methods" are displayed.

As you can see in the above figure, **document.getElementById** function returns a **div element**, whereas the **jQuery** selector returns a **jQuery object**, which is a wrapper around a **div element**. So now, you can call jQuery methods of the jQuery object that is returned by the jQuery selector.

jQuery provides various methods for different tasks, e.g., manipulating the DOM, events, Ajax, etc. The following **table lists different categories of methods**.

Category	Description	Imp Methods
DOM Manipulation	These methods manipulate DOM elements in some manner e.g. changing attribute, style attribute, adding and removing elements etc.	after(), append(), attr(), before(), more..
Traversing	These methods help in navigating from DOM element to another element in a parent child hierarchy e.g. finding ancestors, descendants or sibling element of a specified element.	children(), closest(), each(), first(), next(), filter(), parent(), siblings(), more..
CSS	These methods get and set css related properties of elements.	addClass(), css(), hasClass(), removeClass(), toggleClass() more..
Attributes	These methods get and set DOM attributes of elements.	attr(), html(), removeAttr(), prop(), val(), more..
Events	These methods are used to handle DOM or JavaScript events.	bind(), blur(), change(), click(), dblclick(), focus(), keyup(), keydown(), more..
Effects	These methods are used to add animation to elements.	animate(), fadeIn(), fadeOut(), hide(), show(), stop(), more..

Forms	These methods and event handlers handle forms and their various elements.	blur(), change(), val(), submit(), more..
Ajax	These methods allow Ajax functionalities with jQuery e.g.	get(), getJSON(), post(), load(), more..

- Refer link for jQuery Methods: <https://www.tutorialsteacher.com/jquery/jquery-methods>

jQuery Event.

In most web applications, the **user performs an action to perform an operation**. For example, the user clicks on the save button to save the edited data in a web page. Here, clicking on the button is a user's action, which triggers a click event and a click event handler (function) that saves data.



jQuery Event Methods.

The **jQuery library provides methods to handle DOM events**. Most jQuery methods correspond to native DOM events.

Category	jQuery Method	DOM Event
Form events	blur	onblur
	change	onchange
	focus	onfocus
	focusin	onfocusin
	select	onselect
	submit	onsubmit

Mouse events	click	onclick
	dblclick	ondblclick
	focusout	
	hover	
	mousedown	onmousedown
	mouseenter	onmouseenter
	mouseleave	onmouseleave
	mousemove	onmousemove
	mouseout	onmouseout
	mouseover	onmouseover
	mouseup	onmouseup
	Toggle	

- **Refer link for jQuery Event Methods:** https://www.w3schools.com/jquery/jquery_ref_events.asp
- **Refer link for jQuery Events:** <https://www.tutorialsteacher.com/jquery/jquery-event>

JSON (JavaScript Object Notation)

JSON is a format for storing and transporting data.

JSON is often used when data is sent from a server to a web page.

What is JSON?

- JSON stands for **J**ava**S**cript **O**bject **N**otation
- JSON is a lightweight data interchange format
- JSON is language independent *
- JSON is "self-describing" and easy to understand

* The JSON syntax is derived from JavaScript object notation syntax, but the JSON format is text only. Code for reading and generating JSON data can be written in any programming language.

Where will we use JSON?

Some scenarios include:

- Data storage
- Data configuration and verification
- Generating data structures from user input
- Data transfer from client to server, server to client, and between servers

A note on AJAX — JSON is often used with AJAX, or Asynchronous JavaScript and XML. AJAX is used to transfer data without having to refresh the browser page. Later in this article, we'll look at how to make an AJAX request to a JSON file, and output the content in the browser.

JSON stands for JavaScript Object Notation. It is **a text format that makes it easy to share data between devices, such as Clients and Servers**. Its Origin is based on how JavaScript object works, so in that sense, it is related/close to but not completely a JavaScript Object. Regardless of the fact that it has its origin in JavaScript, it is widely used across many Languages like C, Ruby, Python, and PHP, just to mention a few. Because of the size and ease of converting to a data structure, it is kind of a great alternative to other formats like XML.

JSON Data - A Name and a Value

JSON data is written as name/value pairs.

A name/value pair consists of a field name (in double quotes), followed by a colon, followed by a value:

Example

```
"name": "John"
```

JSON names require double quotes. JavaScript names don't.

JSON - Evaluates to JavaScript Objects

The JSON format is almost identical to JavaScript objects.

In JSON, *keys* must be strings, written with double quotes:

JSON

```
{ "name": "John" }
```

In JavaScript, keys can be strings, numbers, or identifier names:

JavaScript

```
{ name: "John" }
```

JSON Values

In **JSON**, *values* must be one of the following data types:

- a string
- a number
- an object (JSON object)
- an array
- a boolean
- null

In **JavaScript** values can be all of the above, plus any other valid JavaScript expression, including:

- a function
- a date
- undefined

In JSON, *string values* must be written with double quotes:

JSON

```
{ "name": "John" }
```

In JavaScript, you can write string values with double *or* single quotes:

JavaScript

```
{ name: 'John' }
```

Example

```
{ "name": "John", "age": 30, "car": null }
```

JSON objects are surrounded by curly braces {}.

JSON objects are written in key/value pairs.

Keys must be strings, and values must be a valid JSON data type (string, number, object, array, boolean or null).

Keys and values are separated by a colon.

Each key/value pair is separated by a comma.

Accessing Object Values

You can access the object values by using dot (.) notation:

Example

```
myObj = { "name": "John", "age": 30, "car": null };  
x = myObj.name;
```

You can also access the object values by using bracket ([]) notation:

Example

```
myObj = { "name":"John", "age":30, "car":null };  
x = myObj["name"];
```

Looping an Object

You can loop through object properties by using the for-in loop:

Example

```
myObj = { "name":"John", "age":30, "car":null };  
for (x in myObj) {  
    document.getElementById("demo").innerHTML += x;  
}
```

In a for-in loop, use the bracket notation to access the property *values*:

Example

```
myObj = { "name":"John", "age":30, "car":null };  
for (x in myObj) {  
    document.getElementById("demo").innerHTML += myObj[x];  
}
```

Nested JSON Objects

Values in a JSON object can be another JSON object.

Example

```
myObj = {  
  "name": "John",  
  "age": 30,  
  "cars": {  
    "car1": "Ford",  
    "car2": "BMW",  
    "car3": "Fiat"  
  }  
}
```

You can access nested JSON objects by using the dot notation or bracket notation:

Example

```
x = myObj.cars.car2;  
// or:  
x = myObj.cars["car2"];
```

Modify Values

You can use the dot notation to modify any value in a JSON object:

Example

```
myObj.cars.car2 = "Mercedes";
```

You can also use the bracket notation to modify a value in a JSON object:

Example

```
myObj.cars["car2"] = "Mercedes";
```

Delete Object Properties

Use the `delete` keyword to delete properties from a JSON object:

Example

```
delete myObj.cars.car2;
```

Arrays as JSON Objects

Example

```
[ "Ford", "BMW", "Fiat" ]
```

Arrays in JSON are almost the same as arrays in JavaScript.

In JSON, array values must be of type string, number, object, array, boolean or *null*.

In JavaScript, array values can be all of the above, plus any other valid JavaScript expression, including functions, dates, and *undefined*.

Arrays in JSON Objects

Arrays can be values of an object property:

Example

```
{  
  "name": "John",  
  "age": 30,  
  "cars": [ "Ford", "BMW", "Fiat" ]  
}
```

Accessing Array Values

You access the array values by using the index number:

Example

```
x = myObj.cars[0];
```

Looping Through an Array

You can access array values by using a **for-in** loop:

Example

```
for (i in myObj.cars) {  
  x += myObj.cars[i];  
}
```


Or you can use a `for` loop:

Example

```
for (i = 0; i < myObj.cars.length; i++) {  
  x += myObj.cars[i];  
}
```

Nested Arrays in JSON Objects

Values in an array can also be another array, or even another JSON object:

Example

```
myObj = {  
  "name": "John",  
  "age": 30,  
  "cars": [  
    { "name": "Ford", "models": [ "Fiesta", "Focus", "Mustang" ] },  
    { "name": "BMW", "models": [ "320", "X3", "X5" ] },  
    { "name": "Fiat", "models": [ "500", "Panda" ] }  
  ]  
}
```

To access arrays inside arrays, use a `for-in` loop for each array:

Example

```
for (i in myObj.cars) {  
  x += "<h1>" + myObj.cars[i].name + "</h1>";  
  for (j in myObj.cars[i].models) {  
    x += myObj.cars[i].models[j];  
  }  
}
```

Modify Array Values

Use the index number to modify an array:

Example

```
myObj.cars[1] = "Mercedes";
```

Delete Array Items

Use the `delete` keyword to delete items from an array:

Example

```
delete myObj.cars[1];
```

JSON.parse()

A common use of JSON is to exchange data to/from a web server.

When receiving data from a web server, the data is always a string.

Parse the data with `JSON.parse()`, and the data becomes a JavaScript object.

Example - Parsing JSON

Imagine we received this text from a web server:

```
{ "name": "John", "age": 30, "city": "New York" }
```

Use the JavaScript function `JSON.parse()` to convert text into a JavaScript object:

```
var obj = JSON.parse('{ "name": "John", "age": 30, "city": "New York" }');
```

Make sure the text is written in JSON format, or else you will get a syntax error.

Use the JavaScript object in your page:

Example

```
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = obj.name + ", " + obj.age;
</script>
```

JSON From the Server

You can request JSON from the server by using an AJAX request

As long as the response from the server is written in JSON format, you can parse the string into a JavaScript object.

Example

Use the XMLHttpRequest to get data from the server:

```
var xmlhttp = new XMLHttpRequest();
xmlhttp.onreadystatechange = function() {
  if (this.readyState == 4 && this.status == 200) {
    var myObj = JSON.parse(this.responseText);
    document.getElementById("demo").innerHTML = myObj.name;
  }
};
xmlhttp.open("GET", "json_demo.txt", true);
xmlhttp.send();
```

To obtain the JSON, we **use an API called XMLHttpRequest (often called XHR)**. This is a very useful JavaScript object that **allows us to make network requests to retrieve resources from a server via JavaScript** (e.g., images, text, JSON, even HTML snippets), meaning that we can update small sections of content without having to reload the entire page.

JSON from the Server by jQuery.

Definition and Usage

The `getJSON()` method is used to get JSON data using an AJAX HTTP GET request.

Syntax

```
$(selector).getJSON(url,data,success(data,status,xhr))
```

Parameter	Description
<i>url</i>	Required. Specifies the url to send the request to
<i>data</i>	Optional. Specifies data to be sent to the server
<i>success(data,status,xhr)</i>	Optional. Specifies the function to run if the request succeeds Additional parameters: <i>data</i> - contains the data returned from the server. <i>status</i> - contains a string containing request status ("success", "notmodified", "error", "timeout", or "parsererror"). <i>xhr</i> - contains the XMLHttpRequest object

Example

Get JSON data using an AJAX request, and output the result:

```
$("#button").click(function(){
    $.getJSON("demo_ajax_json.js", function(result){
        $.each(result, function(i, field){
            $("#div").append(field + " ");
        });
    });
});
```

The `success` callback is passed the returned data, which is typically a JavaScript object or array as defined by the JSON structure and parsed using the `$.parseJSON()` method. It is also passed the text status of the response.

This is a shorthand Ajax function, which is equivalent to:

```
1 $.ajax({
2   dataType: "json",
3   url: url,
4   data: data,
5   success: success
6 });
```

Data that is sent to the server is appended to the URL as a query string. If the value of the `data` parameter is a plain object, it is converted to a string and url-encoded before it is appended to the URL.

JSON.stringify()

A common use of JSON is to exchange data to/from a web server.

When sending data to a web server, the data has to be a string.

Convert a JavaScript object into a string with `JSON.stringify()`.

Stringify a JavaScript Object

Imagine we have this object in JavaScript:

```
var obj = { name: "John", age: 30, city: "New York" };
```

Use the JavaScript function `JSON.stringify()` to convert it into a string.

```
var myJSON = JSON.stringify(obj);
```

The result will be a string following the JSON notation.

`myJSON` is now a string, and ready to be sent to a server:

Example

```
var obj = { name: "John", age: 30, city: "New York" };
var myJSON = JSON.stringify(obj);
document.getElementById("demo").innerHTML = myJSON;
```

Create JSON string from a JavaScript object.

```
{"name":"John","age":30,"city":"New York"}
```

- Refer link for JavaScript JSON: https://www.w3schools.com/js/js_json.asp

Exercise 1: Practice of the jQuery API.

- Create an HTML5 document “Q1-getJSON.html” in the **Practical9** folder. Write/append the following examples and show the output in the browser.

Load below **JSON file below from the external server.**

URL: <https://mdn.github.io/learning-area/javascript/ojs/json/superheroes.json>

JSON

Sample Output 1 (JSON String)

```
{ "squadName": "Super Hero Squad", "homeTown": "Metro City", "formed": 2016, "secretBase": "Super tower", "active": true, "members": [ { "name": "Molecule Man", "age": 29, "secretIdentity": "Dan Jukes", "powers": ["Radiation resistance", "Turning tiny", "Radiation blast"] }, { "name": "Madame Uppercut", "age": 39, "secretIdentity": "Jane Wilson", "powers": ["Million tonne punch", "Damage resistance", "Superhuman reflexes"] }, { "name": "Eternal Flame", "age": 1000000, "secretIdentity": "Unknown", "powers": ["Immortality", "Heat Immunity", "Inferno", "Teleportation", "Interdimensional travel"] ] }
```

Sample Output 2

Squad Name	Home Town	Formed	Secret Base
Super Hero Squad	Metro City	2016	Super tower

Sample Output.

jQuery.getJSON

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>jQuery getJSON() Method</title>
  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/jquery.min.js"></script>
</head>
<body>
  <h1>JSON</h1>
  <h3>Sample Output 1 (JSON String) </h3>
  <div></div>

  <h3>Sample Output 2</h3>
  <table border="1" id="myTable">
    <thead>
      <tr>
        <th>Squad Name</th>
        <th>Home Town</th>
        <th>Formed</th>
        <th>Secret Base</th>
      </tr>
    </thead>
    <tbody></tbody>
  </table>

  <script>
    $(function () {
      var aPI = "https://mdn.github.io/learning-
area/javascript/ojs/json/superheroes.json";
      $.getJSON(aPI)
        .done(function (data) {
          $('div').html(JSON.stringify(data));
          $('#myTable tbody').append("<tr><td>"
            + data.squadName + "</td><td>" +
            data.homeTown + "</td><td>" +
            data.formed + "</td><td>"
            + data.secretBase + "</td></tr>");
        });
    });
  </script>
</body>
</html>
```

2. Create an HTML5 document “Q1-Ajax.html”. Write/append the following examples and show the output in the browser.

jQuery.ajax()

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>jQuery ajax() Method</title>
  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/jquery.min.js"></script>
</head>
<body>
  <h1>JSON</h1>
  <h3>Sample Output 1 (JSON String) </h3>
  <div></div>

  <h3>Sample Output 2</h3>
  <table border="1" id="myTable">
    <thead>
      <tr>
        <th>Squad Name</th>
        <th>Home Town</th>
        <th>Formed</th>
        <th>Secret Base</th>
      </tr>
    </thead>
    <tbody></tbody>
  </table>

  <script>
    $(function () {
      $.ajax({
        method: "GET",
        url: "https://mdn.github.io/learning-area/javascript/oojs/json/superheroes.json"
      })
      .done(function (data) {

        $('div').html(JSON.stringify(data));

        $('#myTable tbody').append("<tr><td>"
          + data.squadName + "</td><td>" +
          data.homeTown + "</td><td>" +
          data.formed + "</td><td>"
          + data.secretBase + "</td></tr>");
      });
    });
  </script>
</body>
</html>
```


Exercise 2: Practice of the jQuery API.

1. Create an HTML5 document “Q1.html”. Write/append the following examples and show the output in the browser.
 - Visit the website at <https://api.thecatapi.com/v1/images/search> and **learn some of the API endpoints that are available.**
 - **Create the web page that will call the API endpoint, <https://api.thecatapi.com/v1/images/search>, and display the image in the web page.**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Display Image from The Cat API</title>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"
    integrity="sha256-/xUj+3OJU5yExlq6GSYGHk7tPXikynS7ogEvDej/m4="
    crossorigin="anonymous"></script>
</head>

<body>
  <p>
    <img id="result" src="" alt="cat" style="height: 30%; width: 30%;">
  </p>
  <script>
    /*
      $.get("https://api.thecatapi.com/v1/images/search", function(data){
        $("#result").attr("src", data[0].url);
      })
    */
    $.ajax({
      crossDomain: true,
      type: "GET",
      url: "https://api.thecatapi.com/v1/images/search",
      success: function(data, textStatus, jqXHR) {
        $("#result").attr("src", data[0].url);
      }
    })
  </script>
</body>
</html>
```

2. Create an HTML5 document “Q2-Store.html”. Write/append the following examples and show the output in the browser.

Stores the JSON file below in the window.sessionStorage when the “Store JSON” button is clicked. User clicks the “Retrieve JSON file”, will go to a new page to retrieve the JSON file from window.sessionStorage, and sorts the items in descending order by item.

```
let items = [{
  "item": "Downtown",
  "count": "128"
}, {
  "item": "Fells Point",
  "count": "91"
}, {
  "item": "Inner Harbor",
  "count": "89"
}, {
  "item": "Canton",
  "count": "81"
}, {
  "item": "Federal Hill",
  "count": "42"
}, {
  "item": "Mount Vernon",
  "count": "33"
}, {
  "item": "Riverside",
  "count": "29"
}, {
  "item": "Mid-Town Belvedere",
  "count": "29"
}, {
  "item": "Hampden",
  "count": "29"
}, {
  "item": "Charles Village",
  "count": "25"
}, {
  "item": "Little Italy",
  "count": "23"
}, {
  "item": "Greektown",
  "count": "22"
}, {
  "item": "Charles North",
  "count": "22"
}, {
  "item": "Highlandtown",
  "count": "18"
}, {
  "item": "Washington Village",
  "count": "17"
}, {
  "item": "Brooklyn",
  "count": "17"
}, {
  "item": "Carrollton Ridge",
  "count": "16"
}, {
  "item": "Baltimore Highlands",
  "count": "16"
}, {
  "item": "Baltimore-Linwood",
  "count": "15"
}, {
  "item": "Brewers Hill",
  "count": "14"
}];
```

JSON File.



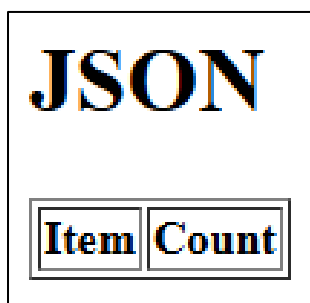
Step 1: Browsing the Page Output.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>JSON</title>
  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/jquery.min.js"></script>
</head>

<body>
  <h1>JSON</h1>
  <button id="myBtn">Store JSON</button>
  <a href="Q2-Retrieve.html">Retrieve JSON file</a>
  <script>
    $(function () {
      let items = [{
        "item": "Downtown",
        "count": "128"
      }, {
        "item": "Fells Point",
        "count": "91"
      }, {
        "item": "Inner Harbor",
        "count": "89"
      }, {
        "item": "Canton",
        "count": "81"
      }, {
        "item": "federal Hill",
        "count": "42"
      }, {
        "item": "mount Vernon",
        "count": "33"
      }, {
        "item": "riverside",
        "count": "29"
      }, {
        "item": "Mid-Town Belvedere",
        "count": "29"
      }, {
        "item": "Hampden",
        "count": "29"
      }, {
        "item": "Charles Village",
        "count": "25"
      }, {
        "item": "Little Italy",
        "count": "23"
      }, {

```

```
        "item": "Greektown",  
        "count": "22"  
    }, {  
        "item": "charles North",  
        "count": "22"  
    }, {  
        "item": "Highlandtown",  
        "count": "18"  
    }, {  
        "item": "Washington Village",  
        "count": "17"  
    }, {  
        "item": "Brooklyn",  
        "count": "17"  
    }, {  
        "item": "Carrollton Ridge",  
        "count": "16"  
    }, {  
        "item": "baltimore Highlands",  
        "count": "16"  
    }, {  
        "item": "Baltimore-Linwood",  
        "count": "15"  
    }, {  
        "item": "Brewers Hill",  
        "count": "14"  
    }  
    }  
};  
  
$('#myBtn').click(function () {  
    if (typeof (Storage) !== "undefined") {  
        sessionStorage.setItem("Items", JSON.stringify(items));  
    }  
  
});  
});  
</script>  
</body>  
</html>
```



Step 2: Click the link “Retrieve JSON file” Output.

3. Create an HTML5 document “Q2-Retrieve.html”. Write/append the following examples and show the output in the browser.

JSON	
Item	Count
Washington Village	17
riverside	29
mount Vernon	33
Mid-Town Belvedere	29
Little Italy	23
Inner Harbor	89
Highlandtown	18
Hampden	29
Greektown	22
Fells Point	91
federal Hill	42
Downtown	128
Charles Village	25
charles North	22
Carrollton Ridge	16
Canton	81
Brooklyn	17
Brewers Hill	14
Baltimore-Linwood	15
baltimore Highlands	16

Step 3: Click the button “Store JSON”, then click the link “Retrieve JSON file” Output.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>JSON</title>
  <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/jquery.min.js"></script>
</head>

<body>
  <h1>JSON</h1>
  <table border="1" id="myTable">
    <thead>
      <tr>
        <th>Item</th>
        <th>Count</th>
      </tr>
    </thead>
    <tbody></tbody>
  </table>
  <script>
    $(function () {
      if (typeof (Storage) !== "undefined") {
        var items = JSON.parse(sessionStorage.getItem("Items"));
        if (items != null) {
          items.sort(function (b, a) {
            if (a.item.toLowerCase() < b.item.toLowerCase()) {
              return -1;
            } else if (a.item.toLowerCase() > b.item.toLowerCase()) {
              return 1;
            } else {
              return 0;
            }
          });

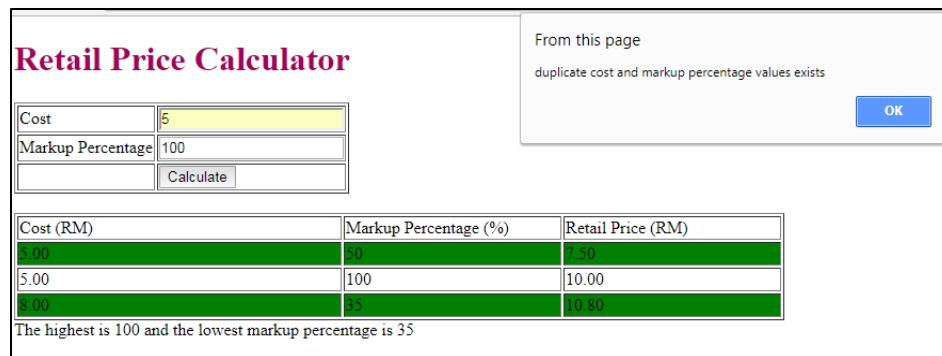
          for (var i = 0; i < items.length; i++) {
            $('#myTable tbody').append("<tr><td>"
              + items[i].item + "</td><td >"
              + items[i].count + "</td></tr>");
          }
        }
      }
    });
  </script>
</body>
</html>
```

Exercise 3: Practice of jQuery API (Student Task).

1. Create a web page that allows the user to input an item's wholesale cost and markup percentage. The program should prevent duplicate cost and markup percentage values and then display the item's retail price in a table format with alternating green color rows. The cost and retail price should be displayed with only two decimal places. Finally, the program should display the highest and lowest markup percentages.

For example:

- If an item's wholesale cost is RM5.00 and its markup percentage is 100 percent, then the item's retail price is RM10.00.
- If an item's wholesale cost is RM5.00 and its markup percentage is 50 percent, then the item's retail price is RM7.50.



Cost (RM)	Markup Percentage (%)	Retail Price (RM)
1.00	50	1.50
5.00	100	10.00
8.00	35	10.80

The highest is 100 and the lowest markup percentage is 35

Sample Output.

2. Create the web page that loads the JSON file in the URL below from the external server and displays the results in a table.

URL: <https://mdn.github.io/learning-area/javascript/oojs/json/superheroes.json>

JSON

Name	Age	Secret Identity
Molecule Man	29	Dan Jukes
Madame Uppercut	39	Jane Wilson
Eternal Flame	1000000	Unknown

Sample Output.